

CODE PROBE

For the Commodore 64

User Manual

```
: D 1000 1021
1000: A2 00 BD 13-10 F0 06 20 00 00 00 00 00 00 00 00
1008: D2 FF E8 D0-F5 20 E4 FF 00 00 00 00 00 00 00 00
1010: F0 FB 00 0D-48 45 4C 4C 00 00 00 00 00 00 00 00
1018: 4F 20 57 4F-52 4C 44 21 00 00 00 00 00 00 00 00
1020: 0D 00 .. ..-.. .. .. .. .. .. .. .. .. ..
0022 (34)

: G 1000
HELLO WORLD!

: █
```


Table of Contents

License	5
1. Introduction	1
Features	1
Version history	1
2. Loading and Starting	3
From Disk	3
What happens at startup	3
Re-entering Code Probe after EXIT	3
3. The Monitor Prompt	5
4. Command Reference	7
4.1 A - Alter Mode	7
4.2 D - Hex Dump	8
4.3 R - Display and Set Registers	9
4.4 RF - Display and Set Register Flags	10
4.5 F - Fill Memory	11
4.6 T - Transfer Memory	12
4.7 G - Execute Program	13
4.8 S - Save File	14
4.9 L - Load File and Directory	15
4.10 CLS - Clear Screen	17
4.11 EXIT - Exit to BASIC	17
5. Error Messages	19
6. The G Command and Shadow Registers	21
How execution works	21
Worked Example: Observing the Shadow Register Round Trip	21
Using BRK as a breakpoint	22

What if the routine never returns?	23
7. Tutorials	25
7.1 Inspecting Memory	25
7.2 Entering a Machine-Language Program	25
7.3 "Hello World" Program	26
7.4 "Hello World" with a BASIC Stub	27
7.5 Saving and Reloading a Program (PRG)	31
7.6 Saving Raw Data (SEQ)	31
7.7 Inspecting Registers After G	32
7.8 Pre-loading Registers Before G	33
7.9 Returning to BASIC and Re-entering	33
8. Tips and Tricks	35
8.1 Working faster at the prompt	35
8.2 Saving, loading, and round-tripping	35
8.3 Running ML safely with G	36
8.4 Living with the 40-column screen	36
Appendix A: Memory Map	37
Appendix B: Quick Reference Card	39
Appendix C: Using Code Probe under VICE Emulation	41
C.1 Setting Up VICE via the GUI	41
C.2 Setting Up VICE via the CLI	41
Appendix D: Architectural Comparison with the VIC-20 Version	43
Glossary	47
Colophon	49

License

Copyright © 2026 Rohin Gosling.

Code Probe and this user manual are distributed under the **MIT License** — a permissive, free-software licence that allows use, modification, and redistribution (including commercial use), provided the copyright notice and licence text are preserved. The full text is at <<https://opensource.org/license/mit/>>.

1. Introduction

Code Probe is a machine-language monitor for the Commodore 64. It is a software-based development tool that runs in the upper-RAM window at **\$C000-\$CFFF** and produces machine-language programs that run on a stock C64 with no additional hardware. Disk I/O uses the standard 1541-style drive at device 8.

The design of Code Probe was inspired by the DOS **DEBUG** utility, and presents a similar terminal-style user interface and commands. All numeric input is hexadecimal: addresses are 4 digits, byte values are 2 digits, and device numbers are 2 digits.

Features

- **Memory inspection** - hex dump with PETSCII character display.
- **Memory editing** - interactive alter mode with cursor navigation and auto-advance.
- **Register management** - view A, X, Y, SP, PC, P, IO and modify any of them; expand the processor status flags as individual bits with **RF**.
- **Memory operations** - fill ranges with a constant byte value, and transfer (copy) ranges to a new location with overlap-safe forward and backward copies.
- **Program execution** - run machine-language programs with the shadow registers loaded into the live CPU; user routines return to the monitor by executing **BRK**, and the CPU state is captured back into the shadow registers.
- **Disk I/O** - save and load PRG and SEQ files to and from a 1541 (or compatible) disk drive, plus a directory listing.
- **Screen control** - clear the display with a single command.
- **Exit to BASIC** - return to BASIC's **READY.** prompt; re-enter Code Probe with **SYS 49152**.

Version history

Year	Version	Platform	Description
1988	1.0	VIC-20	Hand-assembled and poked into RAM with a BASIC machine language loader.
1990	2.0	C64	Ported to C64, with expanded feature set and C64 specific updates.
2026	1.1	VIC-20	Assembly reconstruction of original version 1.0 for the VIC-20.
2026	2.1	C64	Assembly reconstruction of version 2.0 for the C64.

2. Loading and Starting

Code Probe loads at address **\$C000** (49152 decimal) and occupies approximately 4 KiB of RAM in the **\$C000-\$CFFF** region. The address range is permanent RAM on a stock C64, sitting between BASIC ROM (which ends at **\$BFFF**) and the I/O page (which starts at **\$D000**). No cartridge or memory expansion is required.

From Disk

```
LOAD "CODEPROBE",8,1
SYS 49152
```

The **,8,1** parameter loads the program to the address stored in its PRG header (**\$C000**), rather than to BASIC's program area. After loading, **SYS 49152** transfers control to Code Probe.

What happens at startup

1. The screen border and background are set to black via VIC-II registers **\$D020** and **\$D021**.
2. The KERNAL text colour at **\$0286** is set to green (**\$05**).
3. The screen is cleared via **CHROUT \$93**.
4. The title banner is displayed:

```
CODE PROBE (2.1) - ROHIN GOSLING
```

5. A blank line separates the banner from the first prompt.
6. The shadow registers are initialised to default values: **A**, **X**, **Y**, **SP**, and **PC** to **\$00**; **P** to **\$20** (the unused-bit-five default); **IO** to the live **\$01** value at startup (typically **\$37**).
7. The original BRK vector at **\$0316/\$0317** is saved, then replaced with the address of Code Probe's **brk_handler**. This is what allows the **G** command's user routines to return to the monitor by executing a **BRK** instruction (see Section 6).
8. The monitor prompt loop begins.

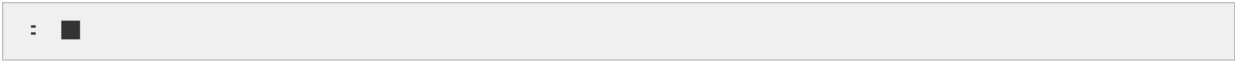
Re-entering Code Probe after EXIT

Once Code Probe has been loaded, **SYS 49152** re-enters it from BASIC's **READY.** prompt. The startup sequence runs again, all input buffers and shadow registers are reset, and the BRK handler is reinstalled. The user's RAM at **\$0801-\$BFFF** is **not** disturbed, so anything entered with **A**, loaded with **L**, or computed by a previous **G** is still there.

A cold reset of the C64 (powering off and on, or hitting the reset switch) zeros the entire 64 KiB of RAM, which loses Code Probe and all user state. Reload the PRG before the next **SYS 49152**.

3. The Monitor Prompt

The monitor prompt is a colon followed by a space:



```
: ■
```

Type a command and press **RETURN** to execute it. After each command completes, a blank line is printed, followed by a new prompt.

All numeric values are entered in **hexadecimal**.

- Addresses are 4-digit hex values (e.g. **1000**, **C000**, **FFFF**).
- Byte values are 2-digit hex values (e.g. **00**, **FF**, **A5**).
- Device numbers are 2-digit hex values (e.g. **08** for the standard primary disk drive).

The C64 keyboard produces uppercase PETSCII by default, so hex digits **A-F** are entered as plain unshifted letters.

The line buffer holds 39 characters, which is the largest line that fits on the 40-column screen with the **:** prompt visible. Typing past 39 characters is silently rejected; the cursor will not advance past column 40.

Command dispatch is **whole-token**: the first space-separated token on the line is matched verbatim against the command table. **R** and **RF** are distinct commands, and **CLS** is a single token (not three letters dispatched on **C**). Token names are case-insensitive — the C64 keyboard's default uppercase mode emits the canonical form, but a name typed in lowercase still matches.

4. Command Reference

Command	Verb	Summary
A	Alter	Write bytes to RAM.
D	Dump	Display memory as hex / PETSCII.
R	Register	Display registers, or set one register's value.
RF	RegFlags	Display registers + flag bit-view, or set one flag.
F	Fill	Fill a memory range with a byte value.
T	Transfer	Copy a memory range to a destination.
G	Go	Execute ML routine; capture registers on BRK.
S	Save	Save a memory range to disk (PRG or SEQ).
L	Load	Load a file from disk, or list a directory.
CLS	Clear	Clear the screen and home the cursor.
EXIT	Exit	Return to BASIC via the warm-start vector.

4.1 A - Alter Mode

Syntax: A <address>

Enters Alter mode - an interactive hex-digit editor for writing bytes to consecutive memory locations. The monitor prints a line prompt showing the current address. The user types hex digits *one at a time* via KERNAL **GETIN** polling. The cursor auto-inserts a separator space after every completed byte and auto-advances to a new prompt line after ten bytes (twenty nibbles, the screen-width budget).

Line prompt format (8 columns: 2-space indent, 4 hex digits, colon, space):

```
  AAAA : █
```

The cursor follows.

Keys:

Key	Action
0-9, A-F	Enter a hex digit. After every second digit, a space is auto-inserted to mark the byte boundary.
RETURN	Commit any complete bytes on the current line and open a new prompt at the new address. On an empty line, exits Alter mode.
Cursor Left	Move one hex-digit slot to the left, hopping over the separator space at byte boundaries. Clamped at the first slot of the current line.
Cursor Right	Move one hex-digit slot to the right, hopping spaces. Clamped at the entry frontier (the next-to-be-typed slot).
INST/DE L	At the frontier, deletes the most recently entered nibble. When the deleted nibble is a low nibble, the trailing separator space is also erased. Ignored when the cursor has been moved back into already-entered data.
Any other key	Silently ignored.

Auto-commit: after the twentieth nibble (tenth byte) is entered on a line, Alter mode commits the bytes to memory, advances the line address by 10, and opens a fresh prompt - without requiring **RETURN**. This keeps every full line within the 40-column screen width.

Exiting Alter mode:

Press **RETURN** on an empty line. A byte-count summary is printed:

```
XXXX (Y)
```

XXXX is the total in hex; **Y** is the decimal count. An odd lone high-nibble at exit time is discarded (only complete bytes are committed).

Example session:

```
: A 1000
1000: A9 41 20 D2 FF 60 00 00 00 00
100A: A9 42
100C:
000C (12)

: D 1000 100B
1000: A9 41 20 D2-FF 60 00 00 A _ -
1008: 00 00 A9 42-.. .. .. B
000C (12)

: ■
```

The first line auto-commits after the twentieth nibble (the second **0** of the trailing **00**). The second line is committed by **RETURN**, and **RETURN** on the empty third line exits Alter mode.

4.2 D - Hex Dump

Syntax: **D** <start> <end>

Displays memory as a hex dump with a PETSCII sidebar. Each row shows the starting address, eight bytes in hex (with a - separator between bytes 3 and 4), and eight printable characters. Non-printable bytes (outside \$20-\$7E and \$A0-\$FE) are displayed as a space glyph.

Output format (40 columns exact):

```
AAAA: HH HH HH HH-HH HH HH HH CCCCCCCC
```

- **AAAA** - address of the first byte in the row.
- **HH** - hex value of each byte. The - between bytes 3 and 4 is a visual aid for scanning long dumps.
- **CCCCCCCC** - character representation of the same eight bytes.

Rows have a 2-space leading indent so each full row, including the indent, is exactly 40 columns.

Parameters:

- **<start>** - 4-digit hex address (required).
- **<end>** - 4-digit hex address (required).

Behaviour:

- Rows are printed from **<start>** to **<end>** inclusive.
- If **<end>** < **<start>**, prints **ERROR: ILLEGAL VALUE**.
- If the dump would read past **\$FFFF**, the operation stops at **\$FFFF**.
- Partial final rows show **..** for missing hex positions and a space for missing PETSCII positions.
- A byte-count summary is printed on a final line:

```
XXXX (Y)
```

where **XXXX** is the count in hex (zero-padded to 4 digits) and **Y** is the count in decimal.

Examples:

```
: D 1000 1010
1000: 00 01 02 03-04 05 06 07
1008: 08 09 0A 0B-0C 0D 0E 0F
1010: 10 .. .. ..-.. .. .. ..
0011 (17)

: D FFFA FFFF
FFFA: 43 FE 48 FF-72 FC .. .. C H _ 
0006 (6)

: █
```

The first dump shows three full rows of eight bytes plus a partial trailing row of one byte. The second dumps the C64's KERNAL reset and IRQ vectors at the top of memory.

4.3 R - Display and Set Registers

Syntax (display): R
 Syntax (set): R <register> <value>

Displays the current shadow register state, or — when given a name and value — sets one register to the supplied value and reprints the register line.

Output format (one line, 39 chars, fits within 40 columns):

```
A:XX X:XX Y:XX SP:XX PC:XXXX P:XX IO:XX
```

Register	Size	Description
A	8-bit	Accumulator.
X	8-bit	X index register.
Y	8-bit	Y index register.
SP	8-bit	Stack pointer.
PC	16-bit	Program counter (last value captured by BRK).
P	8-bit	Processor status register (default \$20).
IO	8-bit	Processor port at \$01 (memory bank configuration).

Validation when setting:

- 8-bit registers (**A**, **X**, **Y**, **SP**, **P**, **IO**) require exactly 2 hex digits.
- The 16-bit register (**PC**) requires exactly 4 hex digits.
- Non-hex characters or wrong-width values produce **ERROR: ILLEGAL VALUE**.

Examples:

```
: R
A:00 X:00 Y:00 SP:00 PC:0000 P:20 IO:37

: R A FF
A:FF X:00 Y:00 SP:00 PC:0000 P:20 IO:37

: R PC 1000
A:FF X:00 Y:00 SP:00 PC:1000 P:20 IO:37

: ■
```

The shadow registers are not the live CPU registers; they are a RAM-resident image that **G** loads into the CPU before each run, and that **brk_handler** repopulates from the CPU when a user routine returns. See Section 6 for the full mechanism.

4.4 RF - Display and Set Register Flags

Syntax (display): **RF**
 Syntax (set): **RF** <flag> <bit>

Displays the register line (same as **R**) followed by the processor status flags expanded as individual bits, or — when given a flag name and a bit value — sets one flag bit and reprints both views.

Output format:


```

A:XX X:XX Y:XX SP:XX PC:XXXX P:XX IO:XX

  NU-BDIZC
P:XXXXXXXX

```

The legend row is indented 4 spaces and the value row 2, so the eight flag characters line up under the eight legend characters.

Flag bit positions (MSB to LSB):

Bit	Flag	Name
7	N	Negative
6	V	Overflow
5	-	(unused, always 1)
4	B	Break
3	D	Decimal
2	I	Interrupt-disable
1	Z	Zero
0	C	Carry

Validation when setting:

- **<flag>** must be exactly one of the eight characters **N**, **V**, **-**, **B**, **D**, **I**, **Z**, **C**.
- **<bit>** must be exactly one character: **0** or **1**. Any other input produces **ERROR: ILLEGAL VALUE**.

Examples:

```

: RF
A:00 X:00 Y:00 SP:00 PC:0000 P:20 IO:37

  NU-BDIZC
P:00100000

: RF C 1
A:00 X:00 Y:00 SP:00 PC:0000 P:21 IO:37

  NU-BDIZC
P:00100001

: RF Z 1
A:00 X:00 Y:00 SP:00 PC:0000 P:23 IO:37

  NU-BDIZC
P:00100011

: █

```

4.5 F - Fill Memory

Syntax: **F** **<address>** **<count>** **<value>**

Fills **<count>** bytes of memory starting at **<address>** with the byte **<value>**.

Parameters:

- **<address>** - 4-digit hex address.
- **<count>** - 4-digit hex count (1 to **\$FFFF**; **\$0000** is a no-op).
- **<value>** - 2-digit hex byte.

Behaviour:

- If **<count>** is **\$0000**, the command silently does nothing.
- If **<address> + <count>** would exceed **\$FFFF**, the count is clamped so the fill stops at **\$FFFF**. The byte-count summary reflects the clamped count, and **ERROR: RAM OVERFLOW** is printed afterwards.
- On completion, prints a byte-count summary:

```
XXXX (Y)
```

Examples:

```
: F 1000 0010 EA
  0010 (16)
: █
```

Fills **\$1000-\$100F** with the 6502 NOP opcode **\$EA**.

```
: F FFF0 0020 00
  0010 (16)
  ERROR: RAM OVERFLOW
: █
```

The user asked for 32 bytes of **\$00** starting at **\$FFF0**, but only the 16 bytes from **\$FFF0-\$FFFF** fit. The summary reports the bytes actually written, and the overflow error follows.

4.6 T - Transfer Memory

Syntax: **T** **<source>** **<count>** **<destination>**

Copies **<count>** bytes from **<source>** to **<destination>**.

Parameters:

- **<source>** - 4-digit hex source address.
- **<count>** - 4-digit hex count.
- **<destination>** - 4-digit hex destination address.

Behaviour:

- The source memory is **not** cleared after the copy; this is a copy, not a move.

- Overlapping ranges are handled correctly: when `<source> >= <destination>` the copy walks forward, byte by byte. When `<source> < <destination>` the copy walks backward, starting at the end of the range. The result is the same as if a temporary buffer had been used.
- If either the source or destination range would exceed `$FFFF`, the count is clamped and **ERROR: RAM OVERFLOW** is printed *after* the byte-count summary.
- On completion, prints a byte-count summary:

```
XXXX (Y)
```

Example:

```
: F 1000 0010 AA
   0010 (16)

: T 1000 0010 2000
   0010 (16)

: D 2000 200F
   2000: AA AA AA AA-AA AA AA AA |||||
   2008: AA AA AA AA-AA AA AA AA |||||
   0010 (16)

: ■
```

Fill 16 bytes at `$1000` with `$AA`, copy them to `$2000`, then verify with a dump.

Overlap example (slide a region forward by one byte):

```
: T 1000 0010 1001
   0010 (16)

: ■
```

Because `<source> < <destination>`, the copy walks backward, so the last byte at `$100F` is moved to `$1010` first, then `$100E` to `$100F`, and so on down to `$1000` ending up at `$1001`. This is the "insert a byte" trick.

4.7 G - Execute Program

Syntax: `G <address>`

Transfers control to the machine-language routine at `<address>` via `RTI` (after loading the shadow registers into the live CPU). The user's routine returns to Code Probe by executing `BRK` (opcode `$00`).

Parameters:

- `<address>` - 4-digit hex start address of the user's routine.

Semantics:

- Before the jump, A, X, Y, SP, P, and the IO byte at `$01` are loaded from the shadow registers. The user's routine sees the exact register state shown by `R`.
- Execution enters via `RTI`, so the user's routine **must** terminate with `BRK` to return to Code Probe — `RTS` does not work, because the stack frame `cmd_g` set up is for `RTI`, not for a JSR-style call.

- Immediately after **BRK**, Code Probe's **brk_handler** captures A, X, Y, SP, the user's P, the user's **\$01**, and PC (adjusted to point at the BRK instruction itself, not past it) into the shadow block.
- The captured state can be inspected with **R** and **RF**. The shadow values persist across multiple **G** invocations and are re-loaded into the CPU on each call.
- If the user's routine fails to **BRK** (for example an infinite loop or a jump to BASIC's warm start), control does not return to Code Probe; the user must press **RUN/STOP** + **RESTORE** or cold-reset the machine.

Example:

```

: A 1000
1000: A9 41 20 D2 FF 00
1006:
0006 (6)

: G 1000
A

: R
A:41 X:00 Y:00 SP:F8 PC:1005 P:30 IO:37

: ■

```

The user's routine loads 'A' into the accumulator, calls KERNAL **CHROUT** (**\$FFD2**), and **BRKs**. The **A** is printed and the **BRK** is captured by **brk_handler** into the shadow register block. Code Probe does not print the registers automatically on **BRK** return — use **R** (or **RF**) to inspect what was captured. **PC** shows **\$1005** — the address of the **BRK** instruction itself.

Self-restart trick: **G C000** re-enters the monitor at its own entry point. The colour / clear / banner sequence re-runs and the user lands at a fresh **:** prompt without leaving the monitor. Because **\$C000** is a normal **G** target, the same RTI/**BRK** trampoline is set up — but the entry sequence's **sei** / **cli** and **BRK**-vector re-installation runs cleanly and produces a fresh banner. Each invocation costs a small amount of stack-frame state that is not strictly recovered, so this is best used as a one-off "reset the screen" shortcut, not in a tight loop.

4.8 S - Save File

Syntax: **S** "<filename>" <device> <start> <end> [<load_address>]

Saves the bytes from <start> to <end> (inclusive) to a file on the named device. The optional <load_address> controls whether the file is written as PRG (with a 2-byte load-address header) or SEQ (raw bytes only).

Parameters:

- <filename> - up to 16 characters of uppercase PETSCII, **enclosed in double quotes**. The quotes disambiguate the filename from the drive's own **,P,W** / **,S,W** syntax that Code Probe constructs internally.
- <device> - 2-digit hex device number (typically **08** for the primary disk drive).
- <start> - 4-digit hex start address.
- <end> - 4-digit hex end address (inclusive).

- **<load_address>** - optional 4-digit hex value. When present, the file is written as PRG with this as its header. When absent, the file is written as SEQ (no header).

Behaviour:

- The drive filename is built with a **,P,W** (PRG, write) or **,S,W** (SEQ, write) suffix so the 1541 records the correct directory type.
- PRG mode writes the 2-byte load address first, then the data bytes.
- SEQ mode writes only the raw data bytes from [**<start>**, **<end>**] — the file contains exactly **end - start + 1** bytes.
- On success, prints:

```

BYTES SAVED:  XXXX (Y)
ADDRESS:      XXXX (Y)           (only if <load_address> was given)

```

Errors:

- Missing or unquoted filename, missing device, missing start, or missing end - **ERROR: ILLEGAL VALUE**.
- Filename longer than 16 characters - **ERROR: ILLEGAL VALUE**.
- **<end>** < **<start>** - **ERROR: ILLEGAL VALUE**.
- Tape error, write-protected disk, disk full, **RUN/STOP** mid-save - **ERROR: SAVE FAILED**. The logical file is closed before the prompt returns.

Example (PRG):

```

: S "PRINTA" 08 1000 1005 1000
  BYTES SAVED: 0006 (6)
  ADDRESS:      1000 (4096)
: █

```

Example (SEQ):

```

: S "DATA" 08 2000 200F
  BYTES SAVED: 0010 (16)
: █

```

The PRG file **PRINTA** reloads with **LOAD "PRINTA",8,1** and is launchable with **SYS 4096**. The SEQ file **DATA** reloads only via Code Probe's **L "DATA" 08 <address>** form (because BASIC's **LOAD** expects a PRG header).

4.9 L - Load File and Directory

The L command supports three modes, distinguished by argument count:

Tokens	Mode	Syntax
2	Directory	<code>L <device></code>
3	PRG load	<code>L "<filename>" <device></code>
4	SEQ load	<code>L "<filename>" <device> <address></code>

4.9.1 Directory Listing

`L <device>` opens the \$ directory channel on the named device and prints each entry as `<filename padded to 16> .<TYPE>`, followed by a file-count summary.

Example:

```
: L 08
CODEPROBE      .PRG
CUBE           .PRG
HELLO          .PRG
DATA           .SEQ
0004 (4)

: █
```

4.9.2 PRG Load

`L "<filename>" <device>` loads a PRG file from the named device into RAM at the address recorded in the file's two-byte header. Code Probe pre-reads the header via `OPEN + CHRIN` to capture the start address (KERNAL LOAD does not preserve it after the fact), then performs the actual load via `KERNAL_LOAD` with secondary address 1.

Output on success:

```
BYTES LOADED: XXXX (Y)
ADDRESS:      XXXX (Y)
```

Example:

```
: L "PRINTA" 08
BYTES LOADED: 0006 (6)
ADDRESS:      1000 (4096)

: █
```

The bytes are placed at \$1000 (the PRG header's load address) and the original byte at \$1000 is overwritten.

4.9.3 SEQ / Raw Load

`L "<filename>" <device> <address>` loads any file as raw bytes, storing them starting at `<address>`. The 2-byte header is **not** consumed; the file is read verbatim. Use this mode for SEQ files saved by `S "<name>" <device> <start> <end>` (no load-address argument), or for relocating a PRG to an address other than its header-recorded start.

Example:

```

: L "DATA" 08 2000
  BYTES LOADED: 0010 (16)
  ADDRESS:      2000 (8192)

: ■

```

4.9.4 Errors

- Missing or unquoted filename, missing device, non-hex device or address - **ERROR: ILLEGAL VALUE.**
- File missing on device, drive error, **RUN/STOP** mid-load, KERNAL returns carry set - **ERROR: FILE NOT FOUND.** The logical file is closed before the prompt returns.

4.10 CLS - Clear Screen

Syntax: **CLS**

Clears the screen and homes the cursor to column 0 of line 0. Emits PETSCII **\$93** via **CHROUT** - the standard KERNAL clear-screen control code. A convenience for wiping a crowded display between experiments.

Parameters: None. Extra tokens are ignored.

Behaviour:

- One **CHROUT \$93** call, no other side effects.
- The monitor's next **:** prompt renders on the freshly cleared top line.

Example:

```

: D 1000 10FF
(many rows of output filling the screen)

: CLS■

```

RETURN

```

: ■

```

4.11 EXIT - Exit to BASIC

Syntax: **EXIT**

Returns to BASIC's **READY.** prompt via the BASIC warm-start vector at **\$A002**. Code Probe stays resident at **\$C000-\$CFFF**; re-entry with **SYS 49152** resumes at a fresh monitor prompt.

Parameters: None. Extra tokens are ignored.

Behaviour:

- The original BRK vector saved at startup (in **original_brk_vector**) is restored to **\$0316/\$0317** so the C64's KERNAL once again handles BRK as it did at cold boot.

- **JMP (\$A002)** jumps through the BASIC warm-start vector. BASIC's warm-start routine resets its own stack pointer and input state, so Code Probe does not need to tidy up beforehand.
- Re-entry via **SYS 49152** re-runs the colour / clear / banner sequence and re-installs the BRK handler. The user's RAM at **\$0801-\$BFFF** is **not** disturbed.

Example:

```
: EXIT  
READY.
```

To return to Code Probe:

```
SYS 49152
```

A cold reset of the C64 zeros all RAM and loses Code Probe; the PRG must be reloaded before another **SYS 49152** will find anything to run.

5. Error Messages

Code Probe defines five error messages. All errors terminate the current command and return the monitor to the prompt.

Message	Cause
ERROR: UNKNOWN COMMAND	The first token of the command line does not match any name in the command table.
ERROR: ILLEGAL VALUE	A non-hex character was entered, a hex value has the wrong width for the target field, an end-address is less than its start, a filename is empty / unquoted / over 16 characters, or the token count on a command line is wrong.
ERROR: RAM OVERFLOW	An F fill or T transfer would extend past \$FFFF . The operation is clamped to the boundary and the bytes that fit are written; the error follows the byte-count summary.
ERROR: FILE NOT FOUND.	An L load (PRG, SEQ, or directory) failed: file missing on device, drive error, or RUN/STOP mid-load.
ERROR: SAVE FAILED	An S save failed: write error, disk full, write-protect tab, or device not present.

The **ERROR: RAM OVERFLOW** message is the only error that can follow a successful (partial) operation. Every other error is reported **before** any work is done.

6. The G Command and Shadow Registers

Code Probe maintains its own copy of the CPU registers in RAM, called the **shadow registers**. The shadow block is an 8-byte array holding A, X, Y, SP, PC (16-bit), P, and IO. It is the user's view of the CPU and is what **R**, **RF**, and **G** operate on.

The shadow registers are *not* the live CPU registers — Code Probe itself uses the CPU to run, and the CPU's actual A / X / Y / SP / P are constantly changing as the monitor's parser, printer, and dispatcher do their work. The shadow registers represent the user program's CPU state, frozen between **G** invocations.

How execution works

1. **Set-up** — The user inspects and (optionally) modifies the shadow registers with **R**, **R <reg> <val>**, and **RF <flag> <bit>**.
2. **G <address>** — **cmd_g** parses the address into **shadow_pc**, disables interrupts, writes **shadow_io** to **\$01**, sets the stack pointer from **shadow_sp**, then pushes **shadow_pc** (high then low) and **shadow_p** onto the stack. It loads A / X / Y from the shadow block, then executes **RTI**. The **RTI** pulls the shadow flags and PC back off the stack, restoring the user's full processor state and dispatching into the user routine.
3. **User routine runs** — The routine sees the exact register state shown by **R**. It can do anything; the only requirement is that it eventually executes a **BRK** (opcode **\$00**) to return.
4. **BRK triggers brk_handler** — The 6510 pushes PC+2 and P, then jumps through the BRK vector at **\$0316/\$0317**, which Code Probe patched at startup to point at **brk_handler**. The handler: - Pulls Y, X, A from the stack (the KERNAL's IRQ frontend pushed these on entry). - Pulls P, PC-low, PC-high (CPU-pushed by the BRK). - Subtracts 2 from the captured PC so it points at the BRK instruction itself. - Captures the user's stack pointer (**tsx**) and the live **\$01** IO byte into the shadow block. - Restores the default memory configuration **\$37** to **\$01**, resets the stack pointer to **\$FF**, restores the screen colours, re-enables interrupts, prints a blank line, and **jmp main_loop** to resume the monitor prompt.
5. **Inspect** — Use **R** to see the captured state. Use **RF** to see the flag bit-view.

Worked Example: Observing the Shadow Register Round Trip

This experiment demonstrates how shadow registers flow into the CPU and back.

Step 1 — Enter a small test program at **\$1000**.

The program loads **\$42** into the accumulator and **\$07** into the X register, then BRKs to return to the monitor:

```
: A 1000
1000: A9 42 A2 07 00
1005:
0005 (5)

: █
```

Hex	Assembly	Description
A9 42	LDA #\$42	Load \$42 into the accumulator.
A2 07	LDX #\$07	Load \$07 into the X register.
00	BRK	Return to Code Probe.

Step 2 — Pre-load the shadow accumulator before execution.

Set the shadow A register to \$FF so you can see it change:

```
: R A FF
A:FF X:00 Y:00 SP:00 PC:0000 P:20 IO:37
: ■
```

Step 3 — Execute the program and inspect the result.

```
: G 1000
: R
A:42 X:07 Y:00 SP:F8 PC:1004 P:20 IO:37
: ■
```

G 1000 runs the routine. The program produces no visible output — it only loads registers and BRKs — so the screen advances to a blank line and the prompt. **R** then displays the captured shadow state. A is now \$42 (set by **LDA #\$42**), X is \$07 (set by **LDX #\$07**), and PC is \$1004 — the address of the BRK instruction.

Step 4 — Verify the shadow state persists.

```
: R
A:42 X:07 Y:00 SP:F8 PC:1004 P:20 IO:37
: ■
```

The shadow registers still hold the values captured at the BRK. You can inspect them, modify them with **R**, and even re-execute the program with **G 1000** — the shadow values will be loaded into the CPU again at the start of each run.

Using BRK as a breakpoint

BRK is a one-byte opcode (\$00). To set a breakpoint inside a larger routine, patch a \$00 over the next instruction at the point of interest, run with **G**, inspect with **R**, then patch the original opcode back and continue.

```
: A 1005
  1005: 00
  1006:
  0001 (1)

: G 1000
A

: R
A:41 X:00 Y:00 SP:F8 PC:1005 P:20 IO:37

: ■
```

The captured **PC** is **\$1005** — the address of the BRK that was patched in. You can then patch the original opcode back and re-execute, or inspect the rest of the registers to understand the state at that point.

What if the routine never returns?

If the user's routine enters an infinite loop or jumps somewhere that never reaches a **BRK**, control does not return to Code Probe. The remedies are:

- **RUN/STOP + RESTORE** - the C64's KERNAL NMI handler restores the default I/O vectors and returns to BASIC's **READY.** prompt. Code Probe stays resident at **\$C000**. Re-enter with **SYS 49152**. Note: **RUN/STOP + RESTORE** does **not** restore the BRK vector; the next entry through **SYS 49152** re-installs it.
- **Cold reset** - last resort; loses Code Probe and any unsaved work because all RAM is zeroed.

7. Tutorials

7.1 Inspecting Memory

To examine the contents of memory at any address range, use the **D** command:

```
: D C000 C01F
C000: A9 00 8D 20-D0 8D 21 D0  7 7 !7
C008: A9 05 8D 86-02 A9 93 20  7  7
C010: D2 FF A9 00-85 0D 85 0E - 7
C018: 85 0F 85 10-85 11 A9 20  7
0020 (32)

: █
```

This displays 32 bytes starting at **\$C000**, which is Code Probe's own entry point. The first byte **\$A9** is the opcode for **LDA #imm**, followed by **\$00** (the immediate value, here a colour byte). The PETSCII sidebar shows that most of these bytes don't render as printable characters and so display as spaces.

Try **D 0400 040F** to peek at the start of screen RAM, or **D E000 E00F** to see the start of the C64's KERNAL ROM.

7.2 Entering a Machine-Language Program

Use the **A** command to enter bytes into RAM. The simplest useful example is a 6-byte routine that prints a single character and returns to the monitor via BRK:

```
: A 1000
1000: A9 41 20 D2 FF 00
1006:
0006 (6)

: █
```

The first line is committed by **RETURN**; the empty **1006:** line exits Alter mode.

The program decodes as:

Hex	Assembly	Description
A9 41	LDA #\$41	Load PETSCII 'A'.
20 D2 FF	JSR \$FFD2	Call KERNAL CHROUT .
00	BRK	Return to Code Probe.

Verify with **D**:

```
: D 1000 1005
1000: A9 41 20 D2-FF 00 .. .. 7A -
0006 (6)

: █
```

Execute with **G**, then inspect the captured register state with **R**:

```

: G 1000
A
: R
A:41 X:00 Y:00 SP:F8 PC:1005 P:20 IO:37
: ■

```

The user's routine prints **A** and BRKs back to the monitor. Code Probe captures the CPU state into the shadow register block but does not display it automatically — **R** is what shows it. **A:41** confirms the accumulator carried 'A' through **JSR \$FFD2**; **PC:1005** is the address of the BRK.

7.3 "Hello World" Program

A meatier example: a 29-byte program that prints **HELLO WORLD!** followed by a blank line, then BRKs to return.

This program is designed to demonstrate a simple machine-language routine that runs comfortably under Code Probe — the final **BRK** returns to the monitor, leaving the **HELLO WORLD!** text on screen above the Code Probe prompt.

Note: If you tried to run the same code from BASIC (for example with **SYS 4096**), the program would execute correctly, but the **BRK** falls through to BASIC's warm-start handler, which clears the screen and prints **READY.** too quickly for the **HELLO WORLD!** text to remain visible. For a Hello World program designed to be launched from BASIC, see Tutorial 7.4 — it adds a keypress wait and a BASIC stub so the program can be started with **RUN** and the output stays on screen long enough to read.

Program listing:

Address	Hex	Assembly	Description
\$1000	A2 00	LDX #\$00	Index = 0.
\$1002	BD 0E 10	LDA \$100E,X	Fetch next char.
\$1005	F0 06	BEQ \$100D	NUL terminator - done.
\$1007	20 D2 FF	JSR \$FFD2	Print char.
\$100A	E8	INX	Advance.
\$100B	D0 F5	BNE \$1002	Loop.
\$100D	00	BRK	Return to Code Probe.
\$100E	0D	-	CR.
\$100F	48 45 4C 4C	-	"HELL"
\$1013	4F 20 57 4F	-	"O WO"
\$1017	52 4C 44 21	-	"RLD!"
\$101B	0D	-	CR.
\$101C	00	-	NUL terminator.

Step 1 — Enter the program.

With auto-commit firing every ten bytes, the Alter-mode transcript is:

```
: A 1000
1000: A2 00 BD 0E 10 F0 06 20 D2 FF
100A: E8 D0 F5 00 0D 48 45 4C 4C 4F
1014: 20 57 4F 52 4C 44 21 0D 00
101D:
001D (29)

: █
```

The first two lines auto-commit after their tenth byte each. The third line of nine bytes is committed by **RETURN**, and **RETURN** on the empty **101D:** prompt exits Alter mode.

Step 2 — Verify with D.

```
: D 1000 101C
1000: A2 00 BD 0E-10 F0 06 20 █ █ █
1008: D2 FF E8 D0-F5 00 0D 48 █ █ █ H
1010: 45 4C 4C 4F-20 57 4F 52 HELLO WOR
1018: 4C 44 21 0D-00 .. .. .. LD!
001D (29)

: █
```

The **HELLO WORLD!** text is visible in the PETSCII sidebar at **\$100F-\$101A**.

Step 3 — Run with G, then inspect the registers with R.

```
: G 1000
HELLO WORLD!

: R
A:00 X:0E Y:00 SP:F8 PC:100D P:32 IO:37

: █
```

The **HELLO WORLD!** line prints and the routine BRKs back to the monitor. Code Probe captures the CPU state into the shadow block on BRK. After executing the **R** command to show the shadow registers, **X:0E** shows the loop terminated when X reached **\$0E** — the offset of the NUL terminator within the data block (a leading CR, twelve characters of **"HELLO WORLD!"**, a trailing CR, and the NUL). **PC:100D** is the BRK address.

7.4 "Hello World" with a BASIC Stub

A more complete Hello World. The program switches the screen to white text on a black background with a black border, clears the screen so the new colour scheme takes effect across the entire display, prints **HELLO WORLD!** followed by a blank line and then **PRESS ANY KEY...**, and finally waits for any keypress before returning to the caller.

It includes a BASIC stub so it can be launched by **RUN** from BASIC's **READY.** prompt as an alternative to **G 080E** from Code Probe. The program lives at the start of the C64's BASIC program area (**\$0801**), the same address BASIC's **RUN** expects to find a tokenised program at, so the entire 100-byte payload behaves like a normal BASIC program from the point of view of **LOAD**, **RUN**, and **LIST**.

Layout:

The program is in two parts:

1. A 13-byte BASIC stub at **\$0801-\$080D** containing the tokenised line **10 SYS 2062**. This is the standard C64 upstart-stub pattern. **RUN** parses the line and dispatches via the **SYS** jump.
2. The ML body at **\$080E-\$0864** (87 bytes), reachable as **SYS 2062** from BASIC or as **G 080E** from Code Probe. The body terminates with **RTS** rather than **BRK** so that BASIC's **RUN**-then-return path works cleanly; from Code Probe, an **RTS** from the user routine returns into BASIC's evaluation loop, not the monitor — so **G 080E** from inside Code Probe is **not** the recommended way to launch this particular program. Use **EXIT** followed by **RUN**, or use the **SYS 2062** that BASIC's stub dispatches.

Program listing:

Address	Hex	Assembly	Description
\$0801	0B 08 0A 00	-	Link to \$080B, line# 10.
\$0805	9E 20	-	SYS token + space.
\$0807	32 30 36 32	-	"2062".
\$080B	00 00 00	-	End of line + end of program.
\$080E	A9 00	LDA #\$00	Black.
\$0810	8D 20 D0	STA \$D020	Border = black.
\$0813	8D 21 D0	STA \$D021	Background = black.
\$0816	A9 01	LDA #\$01	White.
\$0818	8D 86 02	STA \$0286	Text colour = white.
\$081B	A9 93	LDA #\$93	Clear-screen control code.
\$081D	20 D2 FF	JSR \$FFD2	Clear; new cells take colour.
\$0820	A2 00	LDX #\$00	Index = 0.
\$0822	BD 45 08	LDA \$0845,X	Fetch next char of msg1.
\$0825	F0 06	BEQ \$082D	NUL terminator - done.
\$0827	20 D2 FF	JSR \$FFD2	Print char.
\$082A	E8	INX	Advance.
\$082B	D0 F5	BNE \$0822	Loop.
\$082D	A9 0D	LDA #\$0D	Carriage return.
\$082F	20 D2 FF	JSR \$FFD2	Print blank line.
\$0832	A2 00	LDX #\$00	Index = 0.
\$0834	BD 53 08	LDA \$0853,X	Fetch next char of msg2.
\$0837	F0 06	BEQ \$083F	NUL terminator - done.
\$0839	20 D2 FF	JSR \$FFD2	Print char.
\$083C	E8	INX	Advance.
\$083D	D0 F5	BNE \$0834	Loop.

Address	Hex	Assembly	Description
\$083F	20 E4 FF	JSR \$FFE4	KERNAL GETIN - A=key or 0.
\$0842	F0 FB	BEQ \$083F	Spin while no key.
\$0844	60	RTS	Return to caller (BASIC).
\$0845	48 45 4C 4C	-	"HELL"
\$0849	4F 20 57 4F	-	"O WO"
\$084D	52 4C 44 21	-	"RLD!"
\$0851	0D 00	-	CR + NUL terminator.
\$0853	50 52 45 53	-	"PRES"
\$0857	53 20 41 4E	-	"S AN"
\$085B	59 20 4B 45	-	"Y KE"
\$085F	59 2E 2E 2E	-	"Y..."
\$0863	0D 00	-	CR + NUL terminator.

Step 1 — Enter the program.

```

: A 0801
0801: 0B 08 0A 00 9E 20 32 30 36 32
0808: 00 00 00 A9 00 8D 20 D0 8D 21
0815: D0 A9 01 8D 86 02 A9 93 20 D2
081F: FF A2 00 BD 45 08 F0 06 20 D2
0829: FF E8 D0 F5 A9 0D 20 D2 FF A2
0833: 00 BD 53 08 F0 06 20 D2 FF E8
083D: D0 F5 20 E4 FF F0 FB 60 48 45
0847: 4C 4C 4F 20 57 4F 52 4C 44 21
0851: 0D 00 50 52 45 53 53 20 41 4E
085B: 59 20 4B 45 59 2E 2E 2E 0D 00
0865:
0864 (100)

: █

```

Each of the ten lines auto-commits after its tenth byte. **RETURN** on the empty **0865:** prompt exits Alter mode.

Note: Because the first 13 bytes of this program contain the BASIC stub required to allow the program to be run from BASIC using the **RUN** command, it means that if you want to run this program in Code Probe with the **G** command, you need to start execution from address **\$080E** with the command **G 080E**, to ensure that execution starts at the machine language part of the program.

```

: G 080E█

```

RETURN

```
HELLO WORLD!
PRESS ANY KEY...
: ■
```

If you try to execute **G 0801**, or **G 0800**, the program will not run, because the first 13 bytes do not represent valid machine language instructions. In the case of a simple BASIC stub, execution will just pass through back to the Code Probe prompt. However, attempting to execute regions of RAM that don't hold valid machine language instructions will generally lead to unexpected behavior, or even unrecoverable hangs that require restarting the machine.

```
: G 0801
: ■
```

Step 2 — Save to disk.

```
: S "HELLO-RUN" 08 0801 0864 0801
  BYTES SAVED: 0064 (100)
  ADDRESS:    0801 (2049)
```

The five arguments are the filename, device number, start address, end address, and load address. The load address (**\$0801**) is written as a 2-byte PRG header so that **LOAD "HELLO-RUN",8,1** places the data at the correct address.

Step 3 — Run from BASIC.

Restart the machine and load "HELLO-RUN" from disk.

```
LOAD "HELLO-RUN",8,1
SEARCHING FOR HELLO
LOADING
READY.
■
```

If you list the program, you will see the BASIC stub we create between bytes **\$0801** and **\$080D**.

```
LIST
10 SYS 2062
READY.
■
```

The program may be run by executing a regular BASIC **RUN** command. The program may also be run by explicitly executing a **SYS 2062** command, which is equivalent to **G 080E** in Code Probe.

```
RUN■ or SYS 2062■
```

RETURN

```
HELLO WORLD!
PRESS ANY KEY...
READY.
■
```

The screen turns black, then **HELLO WORLD!**, a blank line, and **PRESS ANY KEY...** print starting at the top in white text. The routine spins on KERNAL **GETIN** until a key is pressed, then **RTS** returns to BASIC, which prints **READY.** underneath.

7.5 Saving and Reloading a Program (PRG)

After entering the simple Hello World program from §7.3, save it to disk as a PRG file:

```
: S "HELLO" 08 1000 101C 1000
  BYTES SAVED: 001D (29)
  ADDRESS:    1000 (4096)

: ■
```

The 2-byte PRG header records the load address **\$1000**, so the file is loadable by BASIC on a stock C64 with **LOAD "HELLO",8,1** — note however that **RUN** will not work because there is no BASIC stub; launch with **SYS 4096** instead.

To verify the round trip from Code Probe, fill the program area with zeros and reload:

```
: F 1000 0040 00
  0040 (64)

: L "HELLO" 08
  BYTES LOADED: 001D (29)
  ADDRESS:    1000 (4096)

: G 1000
HELLO WORLD!

: ■
```

The reload places the bytes back at **\$1000** and **G 1000** produces the same **HELLO WORLD!** output as before. As in §7.3, use **R** to inspect the register capture if needed — **A:00 X:0E PC:100D**.

7.6 Saving Raw Data (SEQ)

If you want to save a region of memory without a load-address header — for example a screen capture or a data table — use SEQ mode by omitting the **<load_address>** argument from **S**. The reload then needs an explicit destination address.

Capture the entire 1000-byte screen RAM:

```

: S "SCREEN" 08 0400 07E7
  BYTES SAVED: 03E8 (1000)

: L "SCREEN" 08 0400
  BYTES LOADED: 03E8 (1000)
  ADDRESS:      0400 (1024)

: ■

```

The save captures the entire 40 x 25 screen RAM (1000 bytes) without any header. The reload places it back at **\$0400**, restoring the displayed image.

7.7 Inspecting Registers After G

The **R** command shows what the CPU looked like at the instant the most recent **G** command's user routine executed **BRK**. Code Probe does not print the register state automatically on BRK return — **R** (or **RF**) is what produces the register line.

To see this in action, enter a small routine that loads known values:

```

: A 1100
  1100: A9 42 A2 07 A0 09 00
  1107:
  0007 (7)

: ■

```

The bytes decode as:

Hex	Assembly	Description
A9 42	LDA #\$42	Load \$42 into accumulator.
A2 07	LDX #\$07	Load \$07 into X.
A0 09	LDY #\$09	Load \$09 into Y.
00	BRK	Return.

Run the program — the routine produces no visible output, only register changes — then use **R** and **RF** to inspect the captured state:

```

: G 1100

: R
A:42 X:07 Y:09 SP:F8 PC:1106 P:20 IO:37

: RF
A:42 X:07 Y:09 SP:F8 PC:1106 P:20 IO:37

  NV-BDIZC
  P:00100000

: ■

```

The shadow registers persist until the next **G** runs. Run another routine, and they update. The **RF** view shows that the only flag set is bit 5 — the unused "always 1" bit pushed by the BRK / RTI sequence — which is the typical "no flags raised" P value a clean routine leaves behind.

7.8 Pre-loading Registers Before G

The **R** and **RF** commands let you set up specific register state before running a routine. This is useful for testing routines that read their inputs from A / X / Y rather than from memory.

For example, a routine that prints the character in A:

```
: A 1200
1200: 20 D2 FF 00
1204:
0004 (4)

: █
```

Set A to '!' (PETSCII \$21), run, then inspect with **R**:

```
: R A 21
A:21 X:00 Y:00 SP:00 PC:0000 P:20 IO:37

: G 1200
!

: R
A:21 X:00 Y:00 SP:F8 PC:1203 P:20 IO:37

: █
```

The **!** prints, the BRK fires, and the registers come back unchanged (except for SP and PC, which are managed by the call mechanics).

Repeat with a different value:

```
: R A 23
A:23 X:00 Y:00 SP:F8 PC:1203 P:20 IO:37

: G 1200
#

: R
A:23 X:00 Y:00 SP:F8 PC:1203 P:20 IO:37

: █
```

This pattern — patch register, run, observe — is the closest the monitor gets to an interactive debugger.

7.9 Returning to BASIC and Re-entering

To leave Code Probe without losing it from memory:

```
: EXIT
READY.
```

Now you are at BASIC's prompt. You can run BASIC programs, type calculator expressions, or do anything BASIC does. Code Probe is still in RAM at \$C000. To go back:

```
SYS 49152
```

Code Probe re-runs its startup sequence — screen colours, clear, banner, prompt, BRK-handler installation — but the user's RAM at **\$0801-\$BFFF** is untouched. Anything you entered there with **A**, or loaded with **L**, is still present.

A cold reset (powering the C64 off and on, or hitting the reset switch) zeros all RAM, which loses Code Probe. Reload it from disk before the next **SYS 49152**.

8. Tips and Tricks

This section collects the small body of expert-user wisdom that is not obvious from the command syntax alone — shortcuts, workarounds, and rough edges that experienced users tend to learn either by reading the source or by burning themselves once. None of it is required reading; the sections above cover everything you strictly need to drive Code Probe. But the first time one of these tips saves you an hour of head-scratching, you will be glad it was written down.

8.1 Working faster at the prompt

- **Whole-token dispatch** - Command names are matched verbatim. **R** is the register command, **RF** is the flag command, and there is no shortcut like **D** for "display registers" — that's already the dump command. When in doubt, type the full canonical name.
- **Auto-commit in Alter mode** - After twenty hex digits (ten bytes) on an Alter row, the monitor commits the line and opens a fresh prompt at the next address automatically; no **RETURN** needed. On long edits you can hold the keys down and just keep typing through the entire region. **RETURN** is only required for partial rows of fewer than ten bytes, and to exit Alter mode entirely.
- **G \$C000 as a soft reset** - Re-entering the monitor at its own start address re-runs the banner, screen colours, clear sequence, and BRK-vector installation. It is a one-shot way to reset visible screen state mid-session without exiting to BASIC. Best used occasionally, not in a loop.
- **The 39-character input limit** - The line buffer leaves one column for the **:** prompt on the 40-column screen. Plan command lines accordingly. Very long Alter regions are easier as multiple **A** invocations than as a single overflowing **A** line, but in practice a single **A <addr>** invocation lasts as many lines as you keep typing — only the *command line itself* is limited.

8.2 Saving, loading, and round-tripping

- **PRG vs SEQ — when to use which** - PRG saves prepend a 2-byte load-address header; reload with **L "name" <device>** (or BASIC's **LOAD "name",8,1**) and the file goes back to the address it came from, ready to **SYS** directly or to launch with **RUN** if it has a BASIC stub. SEQ saves are raw bytes with no header; reload requires you to specify the destination address explicitly with **L "name" <device> <address>**. Use PRG for self-contained programs you intend to re-run; use SEQ for data payloads or whenever you need to relocate the bytes to a different address on reload.
- **The round-trip workflow** - Author and debug your routine in Code Probe at any address you like, save it as PRG with **S** and the load address you actually want, then **EXIT** and run with **LOAD "name",8,1** followed by **SYS <decimal-address>** from BASIC. Or embed a BASIC stub at **\$0801** and use **RUN**.
- **Code Probe can save a copy of itself** - The monitor image spans **\$C000-\$CFFF**. **S "CODEPROBE" 08 C000 CFFF C000** saves the whole 4 KiB block as a PRG. The resulting file reloads on any C64 with **LOAD "CODEPROBE",8,1** followed by **SYS 49152** to launch. Useful for capturing a known-good build before experimenting with a patched copy in RAM. Note that the monitor's working state (input buffer, shadow registers) is part of the saved image; the re-entry sequence resets all of it, so a self-save at any prompt produces a usable PRG.
- **Filenames in S and L need quotes** - Unlike BASIC's **LOAD** command (which uses unquoted names in **LOAD "..."** form), Code Probe's **S** and **L** require the filename in **"<name>"** form. The quotes disambiguate the user's name from the drive's **,P,W / ,S,W** syntax that Code Probe constructs internally.

- **Exit and re-enter for a clean slate** - **EXIT** returns to BASIC's **READY.** prompt; **SYS 49152** re-enters Code Probe with a fresh banner. Anything in RAM at **\$0801-\$BFFF** survives the round trip — this is a useful way to reset the monitor's internal state (input buffer, shadow registers) without losing work.

8.3 Running ML safely with G

- **Your routine must end in BRK, not RTS or RTI** - Code Probe enters routines via **RTI** through a stack frame containing the user's PC and P; only **BRK** returns control cleanly via the **\$0316** vector that **brk_handler** is patched into. **RTS** will pull garbage off the stack (the RTI frame, not a JSR return address). **RTI** will pull state and immediately jump again, usually somewhere unhelpful.
- **Inspect registers as you go** - Pre-load the shadow registers with **R <reg> <val>** and **RF <flag> <bit>** before each **G**. The routine sees the exact state you set up, BRKs back, and the captured state is visible with the next **R** or **RF**.
- **BRK as a one-byte breakpoint** - Patch a **\$00** over the next instruction at the point of interest, run with **G**, inspect with **R** and **RF**, then patch the original opcode back. The captured PC always points at the BRK address itself (the handler subtracts the 2-byte BRK signature offset for you).
- **Decimal mode survives across BRK** - Unlike the VIC-20 sibling, the C64 **brk_handler** does not issue **CLD** after capturing the user's flags. Instead, it restores the default banking byte **\$37** to **\$01**, which has no effect on the D flag. If your routine sets decimal mode and BRKs without clearing it, the captured P will show **D=1**, and Code Probe's monitor arithmetic continues to work because the prompt loop runs after the handler restores **\$37** and resets SP — but if you **G** again with that captured P, the user's routine starts in decimal mode. Use **RF D 0** to clear it.

8.4 Living with the 40-column screen

- **The 40-character wrap quirk** - Writing exactly 40 characters to the C64 screen auto-wraps the cursor as a side effect of the final **CHROUT**. A subsequent explicit carriage return then advances the cursor a *second* time, leaving a spurious blank row between outputs. Code Probe's full dump rows are exactly 40 columns wide and intentionally omit the trailing carriage return for this reason — the natural auto-wrap moves the cursor to the next line, and a partial final row gets an explicit CR because it doesn't reach column 40. If you write your own screen-output routines, follow the same pattern: skip the **CR** after any row that is exactly 40 characters wide.
- **Why dump rows include the visual -** - The **-** between bytes 3 and 4 in a dump row is a scanning aid, not a structural separator. When reading a long dump it makes the byte-index easy to count by halves: bytes 0-3 to the left of the **-**, bytes 4-7 to the right.
- **The 40 column screen is generous** - Compared to the VIC-20's 22-column screen, the C64 has plenty of room. The full register line **A:XX X:XX Y:XX SP:XX PC:XXXX P:XX IO:XX** is 39 characters — fits with one column to spare. Long filenames (up to 16 chars) fit comfortably alongside the rest of the **S** command tokens. There is no **FILE:** second-line prompt as on the VIC-20; everything fits on one line.

Appendix A: Memory Map

Code Probe resides at **\$C000-\$CFFF**, the 4 KiB upper-RAM window between BASIC ROM and the I/O page. Programs developed under Code Probe typically live in BASIC's program area (**\$0801-\$9FFF**) or in the smaller free regions adjacent to Code Probe.

\$FFFF	KERNAL ROM
\$E000	I/O Registers (VIC, SID, CIA)
\$D000	Color RAM (\$D800-\$DBFF)
\$C000	CODE PROBE 2.1 (\$C000-\$CFFF)
\$B000	BASIC ROM
\$A000	Free RAM (user programs and data)
\$0801	BASIC program area starts
\$0800	Screen RAM (1000 bytes)
\$0400	KERNAL / BASIC work area
\$0200	6502 hardware stack
\$0100	Zero page
\$0000	

Safe areas for user programs: **\$0801-\$9FFF** (about 38 KiB of free RAM when BASIC programs are not present and BASIC RAM is fully banked in). Programs entered here can be saved with PRG headers and loaded back on a stock C64 with **LOAD "<name>",8,1**.

Avoid writing to:

- **\$C000-\$CFFF** — Code Probe lives here.
- **\$D000-\$DFFF** — VIC-II, SID, CIA, and color RAM. Writing to these is sometimes intentional (graphics, sound) but mostly unintended.
- **\$A000-\$BFFF** — BASIC ROM (banked in by default). Writes go to underlying RAM but reads return ROM contents until banking is changed via **\$01**.
- **\$E000-\$FFFF** — KERNAL ROM (banked in by default). Same as BASIC ROM regarding writes vs. reads.

The processor port at **\$01** controls which ROMs are banked in. The default value **\$37** enables BASIC, KERNAL, and I/O. Code Probe preserves and restores **\$37** across **G** invocations via the IO shadow register.

Appendix B: Quick Reference Card

Cmd	Syntax	Description
A	A <address>	Enter Alter mode to write hex bytes to RAM. 10 bytes per line auto-commit; RETURN to commit partial; empty RETURN to exit.
D	D <start> <end>	Hex dump memory from start to end (inclusive). 8 bytes per row with PETSCII sidebar.
R	R	Display all shadow registers.
R	R <register> <value>	Set a shadow register and reprint.
RF	RF	Display registers + flag bit-view.
RF	RF <flag> <0 1>	Set a processor status flag bit and reprint.
F	F <address> <count> <value>	Fill memory with a byte value. Clamps at \$FFFF and reports overflow.
T	T <source> <count> <destination>	Copy memory from source to destination. Overlap-safe.
G	G <address>	Execute ML routine at <address>. User routine must end in BRK. Captures A/X/Y/SP/PC/P/IO.
S	S "<file>" <device> <start> <end> [<load_addr>]	Save memory to file. With load address = PRG; without = SEQ.
L	L <device>	List files on device.
L	L "<file>" <device>	Load PRG file (uses file's load address).
L	L "<file>" <device> <address>	Load SEQ file to specified address.
CLS	CLS	Clear screen and home cursor.
EXIT	EXIT	Return to BASIC (READY. prompt). Re-enter with SYS 49152.

All address and count values are hexadecimal. Addresses are 4 digits, byte values are 2 digits, device numbers are 2 digits (typically 08 for the primary disk drive). Filenames are enclosed in double quotes, up to 16 characters of uppercase PETSCII.

Appendix C: Using Code Probe under VICE Emulation

The preceding sections of this manual describe Code Probe's behaviour on real C64 hardware with a 1541 (or compatible) disk drive at device 8. The VICE emulator can host the same software stack on a desktop computer, with very little setup needed because the C64 configuration that Code Probe targets is the unmodified stock machine.

This section describes the two configuration paths — the GUI-driven settings dialogs in the VICE menu bar, and the equivalent command-line invocation. Once VICE is configured, the rest of this manual applies unchanged.

Unlike the VIC-20 sibling project, no cartridge ROM needs to be sourced. Code Probe is pure 6502 assembly that occupies normal C64 RAM at `$C000-$CFFF`; the C64 has the right RAM at the right address by default.

C.1 Setting Up VICE via the GUI

1. **Launch x64sc.** From the VICE installation, run `x64sc` (the accurate-cycle C64 emulator). On Windows it is typically `x64sc.exe` under the `bin` folder of the WinVICE install; on Linux and macOS it is on the default `PATH` after a package-manager install.
2. **Confirm machine model.** From the menu bar:

```
Settings -> Machine -> Model
```

Choose `C64 PAL` or `C64 NTSC` to match the display you are used to. Both work for Code Probe; the only practical difference is the screen refresh rate.

3. **Optional — attach a disk image.** If you want to save and load files inside the emulator:

```
File -> Attach disk image -> Drive 8
```

Select a `.d64` image (or create one via `File -> Create disk image`). The emulator then presents that disk to Code Probe at device 8, just like a 1541.

4. **Load Code Probe.** Use `File -> Smart attach disk/tape image` and select `codeprobe.prg`. VICE detects the PRG load address and offers to autostart; accept. The C64 boots, types `LOAD "*",8,1` followed by `SYS 49152`, and the Code Probe `:` prompt appears.

Alternatively, load the PRG manually after boot:

```
LOAD "CODEPROBE",8,1
SYS 49152
```

5. **Verify Code Probe is alive.** At the `:` prompt, type `D C000 C00F`. The dump should show the first 16 bytes of the monitor's own code — the LDA / STA / colour-setup sequence at the entry point.

C.2 Setting Up VICE via the CLI

The same configuration can be expressed as a single `x64sc` command, which is more reproducible than the GUI path because it does not depend on saved emulator preferences:

```
x64sc -autostart <path-to-codeprobe.prg>
```

The flags do the following:

- **-autostart <path>** — loads the supplied PRG on boot, honouring its load-address header (so **\$C000** is loaded directly), then types **SYS 49152** at the BASIC prompt. The Code Probe **:** prompt appears within a second or two of the cold-boot banner.

To attach a disk image at the same time:

```
x64sc -8 <path-to-disk.d64> -autostart <path-to-codeprobe.prg>
```

The **-8 <image>** flag attaches a disk image to drive 8. Inside Code Probe, **L 08** lists the disk's directory.

The **x64sc** binary lives inside the VICE installation directory: on Windows it is typically **x64sc.exe** under the **bin** folder of the WinVICE install, and on Linux and macOS it is on the default **PATH** after a package-manager install.

Appendix D: Architectural Comparison with the VIC-20 Version

Code Probe exists in two versions: version 1.1 for the VIC-20 and version 2.1 for the Commodore 64. Both are 6502 assembly programs sharing the same parser conventions, alter-mode cursor handling, and command-table dispatch — but the underlying machines differ enough that the two builds make different design choices in several areas. The table below summarises the architectural deltas at a glance, useful when porting routines between the two builds or when reading the source of one version with experience of the other.

Aspect	VIC-20 v1.1	C64 v2.1
Host machine	Commodore VIC-20	Commodore 64
CPU	MOS 6502	MOS 6510
Load address	\$0480 (SYS 1152)	\$C000 (SYS 49152)
Code size budget	≤ 2944 bytes (\$0480-\$0FFF)	≤ 4096 bytes (\$C000-\$CFFF)
Cartridge required	Yes — VIC-1211A Super Expander	None — pure software, stock C64
Reason for cartridge	Provides 3 KiB RAM at \$0400 to host CP	N/A — \$C000-\$CFFF is permanent stock RAM
User ML buffer	\$1001-\$1DFF (~3.5 KiB)	\$0801-\$BFFF (~46 KiB)
Screen dimensions	22 columns × 23 rows	40 columns × 25 rows
Screen RAM	\$1E00-\$1FF9	\$0400-\$07E7
Color RAM	\$9400 (high-byte offset +\$78)	\$D800 (high-byte offset +\$D4)
Video chip	VIC-I	VIC-II
Border / background register	\$900F (combined nibble pair)	\$D020 / \$D021 (separate)
Storage device	Tape (datasette, device 1)	Disk (1541 / SD2IEC, device 8)
Filename quoting in S / L	Unquoted (e.g. L CUBE)	Double-quoted (e.g. L "CUBE" 08)
Filename type suffix on save	N/A	,P,W for PRG / ,S,W for SEQ
S command PRG mechanism	KERNAL_SAVE (\$FFD8)	OPEN + CHKOUT + per-byte CHROUT
S command load-address override	No (save addr = in-memory addr)	Yes (optional <load_address> argument)
S filename input	Two-line: separate FILE: prompt	Inline as token 1 of S command
Number of commands	8	11
Command set	A, CLS, D, EXIT, G, L, R, S	A, CLS, D, EXIT, F, G, L, R, RF, S, T
Commands unique to this build	—	F, T, RF, R-set form, RF-set form
Command dispatch	Single-character (first letter)	Whole-token (R and RF distinct)

Aspect	VIC-20 v1.1	C64 v2.1
Input-buffer line width	19 chars (22-col screen budget)	39 chars (40-col screen budget)
Alter mode bytes per line	5 (10 nibbles)	10 (20 nibbles)
Dump row format	4 bytes + 4 PETSCII (22 cols, no indent)	8 bytes + - sep + 8 PETSCII (40 cols, indented)
G command entry	JSR through self-modifying trampoline	RTI through stack frame (loads shadow registers)
G command return	User routine ends with RTS	User routine ends with BRK (\$00)
BRK vector handling	Not installed	Patched at \$0316/\$0317 to brk_handler
Shadow registers	Read-only capture (A, X, Y, P)	R/W: A, X, Y, SP, PC, P, IO (loaded into CPU)
R command set form	No (display only)	Yes (R <register> <value>)
Decimal-mode handling on G	CLD after RTS , before flag capture	User's D flag preserved; banking byte reset to \$37
Banner	Two lines (title + author)	Single line
Color theme	White on black	Green on black
Distinct error messages	3 (UNKNOWN / ILLEGAL / FILE)	5 (UNKNOWN / ILLEGAL / RAM OV / NOT FOUND / SAVE)
BASIC warm-start vector	\$C002	\$A002
BASIC upstart stub at PRG start	Yes (10 SYS 1152 at \$0401)	None (avoids ~50 KiB PRG zero-fill gap)
MEMTOP protection from BASIC	Recommended (POKE 55,0 : POKE 56,16)	Not needed (monitor sits above BASIC RAM)
Cube fixture footprint	\$1001-\$1DD4 (~2 KiB unexpanded VIC-20)	\$0801-\$17EB (~4 KiB stock C64)

A handful of these rows carry most of the architectural weight:

- **Cartridge requirement.** The VIC-20 has no useful free RAM window for a 3 KiB monitor on a stock machine, so it leans on the Super Expander cartridge's 3 KiB at **\$0400**. The C64 has permanent RAM at **\$C000-\$CFFF** between BASIC ROM and the I/O page, so no cartridge is needed.
- **G command return mechanism (RTS vs BRK).** This is the largest code-level divergence between the two builds. The **RTI/BRK** trampoline buys two-way register flow — shadow registers loaded into the CPU before the jump, live CPU state captured back into the shadow block on **BRK** — at the cost of a vector patch and a dedicated **brk_handler**. The VIC-20 stays simpler with a **JSR/RTS** trampoline and one-way register capture (no register pre-load).
- **Storage device.** Tape vs disk drives the entire S/L command shape. Tape uses unquoted filenames, secondary-address conventions (SA = 0/1) per direction, and **KERNAL_SAVE** for PRG writes. Disk uses double-quoted filenames, drive-side type suffixes (**,P,W / ,S,W**), and an explicit **OPEN/CHKOUT/CHROUT** byte-loop for SAVE so that the declared load address can differ from the in-memory start.
- **No BASIC stub at \$0801 on the C64.** The C64 build deliberately ships without the **10 SYS 49152** upstart line that the VIC-20 build uses at **\$0401**. Adding one to the C64 PRG would force the file to span

\$0801-\$CFFF on disk (~50 KiB, almost all zeros), in exchange for letting **RUN** work after a **LOAD**. The trade-off goes the other way on the VIC-20 because the stub at **\$0401** is right next to the monitor at **\$0480** — there's no gap to zero-fill.

- **MEMTOP protection.** On the VIC-20, the user ML buffer at **\$1001-\$1DFF** is also BASIC's program area, so a careless **RUN** from BASIC could grow variables into the user's machine code. The recommended **POKE 55,0 : POKE 56,16 : CLR** lowers BASIC's MEMTOP and protects the buffer. On the C64 the monitor sits above BASIC ROM at **\$C000**, well clear of BASIC's variable space, so no protection is needed.
-

Glossary

Compact reference for the platform-specific and typesetting terms used throughout this manual. Definitions are deliberately brief — for full background on the C64's hardware and KERNAL conventions, consult the *Commodore 64 Programmer's Reference Guide*.

- **Alter mode** — Code Probe's interactive byte-edit mode entered by the **A** command. Accepts hex nibbles directly without an **Enter** keystroke per byte; commits on **Enter** at end-of-line or on **Run/Stop** to abort.
- **BRK** — The 6502 software-break instruction (**\$00**). Pushes flags and the return address onto the stack and jumps through the IRQ vector. Code Probe patches the BRK vector at **\$0316** / **\$0317** so user routines can return to the monitor by executing **BRK**.
- **C64** — Commodore 64. The 8-bit home computer (1982) Code Probe runs on.
- **Colour RAM** — The VIC-II's 1 KiB block at **\$D800** holding the foreground-colour low nibble for each of the 1000 screen cells.
- **CHRIN** / **CHROUT** — KERNAL routines for character input (**\$FFCF**) and output (**\$FFD2**). Code Probe uses these for terminal-style I/O.
- **Dispatch table** — Code Probe's internal mapping from command tokens (**A**, **D**, **R**, **RF**, ...) to subroutine addresses. The table is whole-token, so **R** and **RF** are distinct commands.
- **Folio** — The printed page number on a typeset page. Code Probe's PDF manual prints a folio on every body page from chapter 1 onward; front-matter pages (cover, TOC, License) carry no folio.
- **Hex nibble** — A 4-bit value (**0-F**). Two nibbles make one byte. Code Probe's **A**-command input is line-buffered as nibbles, two per byte.
- **KERNAL** — The C64's built-in operating-system ROM at **\$E000-\$FFFF**. Exposes routines (CHROUT, CHRIN, OPEN, SETLFS, ...) via fixed entry-point addresses.
- **Monitor prompt** — Code Probe's **:** prompt, where commands are entered. The whole-token parser treats the line as a command plus space-separated arguments.
- **Orphan** — In typography, the first line of a paragraph stranded alone at the bottom of a page, separated from the rest of its paragraph overleaf.
- **PETSCII** — Commodore's 8-bit character encoding. Overlaps ASCII for **\$20-\$5F** (digits, upper-case letters, punctuation) but diverges in the **\$60-\$7F** range (graphics characters in PETSCII vs lower-case in ASCII).
- **Recto** — In bookbinding, the right-hand page of a spread (odd-numbered). Code Probe's chapters and front-matter sections always open on a recto page; a blank verso is inserted before the chapter when needed to enforce this.
- **RTI** — The 6502 return-from-interrupt instruction (**\$40**). Pulls the processor status register and program counter from the stack. Code Probe's **G** command builds an **RTI** frame on the stack so it can load the user's full register set (A, X, Y, P, PC) before jumping to user code.
- **Screen RAM** — The VIC-II's 1 KiB block at **\$0400** holding the screen-code byte for each of the 1000 screen cells.
- **Shadow registers** — Code Probe's persistent register snapshot block. The **R** and **RF** commands display and modify these; the **G** command loads them into the CPU before jumping to user code, and the BRK handler captures the live CPU state back into them on return.
- **Verso** — In bookbinding, the left-hand page of a spread (even-numbered). The opposite of recto.

- **Widow** — In typography, the last line of a paragraph stranded alone at the top of a page, separated from the rest of its paragraph on the page before.
 - **Zero page** — The first 256 bytes of memory ([\\$00-\\$FF](#)). Accessed by 6502 instructions in two bytes instead of three, making zero-page reads and writes faster than reads and writes elsewhere. Code Probe's parser, line buffer, and 16-bit pointers live in zero page.
-

Colophon

The C64 graphics characters in this manual's **D**-command dump examples are typeset with **C64 Pro Mono** by **Style64**, available at <<http://style64.org/c64-truetype>>.

